

PROJECT REPORT

SpendWise

Project Title: SpendWise

Team ID: SWTID-2026-9229

Team Leader: KAUSHIKA A

Team Members:

JENITA G

JOTHIKA L

KAVINISHA H

YAMINI S

SpendWise

Project Title

SpendWise – A Simple Backend Learning Project using Nodejs, Express, MongoDB, and JWT

Project Overview

SpendWise is a backend application designed to help beginner developers learn how to build secure REST APIs using Nodejs, Express, MongoDB, and JWT authentication. The project focuses on implementing user authentication and basic CRUD operations in a clean and simple way so students can understand how backend systems work in real-world scenarios.

Key Objectives:

- Provide secure user authentication using JWT
- Enable basic CRUD operations on user data (transactions/records)
- Demonstrate MongoDB integration using Mongoose
- Show how protected routes work using middleware
- Maintain a simple and beginner-friendly backend architecture

Target Users:

- Beginner backend developers
- Students learning Nodejs and Express
- Freshers preparing for technical interviews
- Anyone who wants a simple project to understand JWT and MongoDB
- Learners who want hands-on practice with REST APIs

Project Instruction

SpendWise is a backend-only project built using Nodejs and Express.

Quick Start:

`npm init`

`npm install`

`npm run dev`

Steps to Create the Project:

- Choose or create a directory for the backend project.
- Open terminal and navigate to that directory.
- Initialize Node.js project using `npm init`.
- Install required packages: `express`, `mongoose`, `jsonwebtoken`, `bcrypt`, `cors`, `dotenv`.
- Create folders: `models`, `routes`, `controllers`, `middleware`, `config`.
- Start the server using `npm run dev`.
- Test APIs using Postman or Thunder Client at <http://localhost:5000>.

Pre-requisites:

Software Requirements

Software	Purpose
Node.js	JavaScript runtime for backend
MongoDB	NoSQL database for data storage
npm	Package manager
Git	Version control system
Code Editor	VS Code or similar

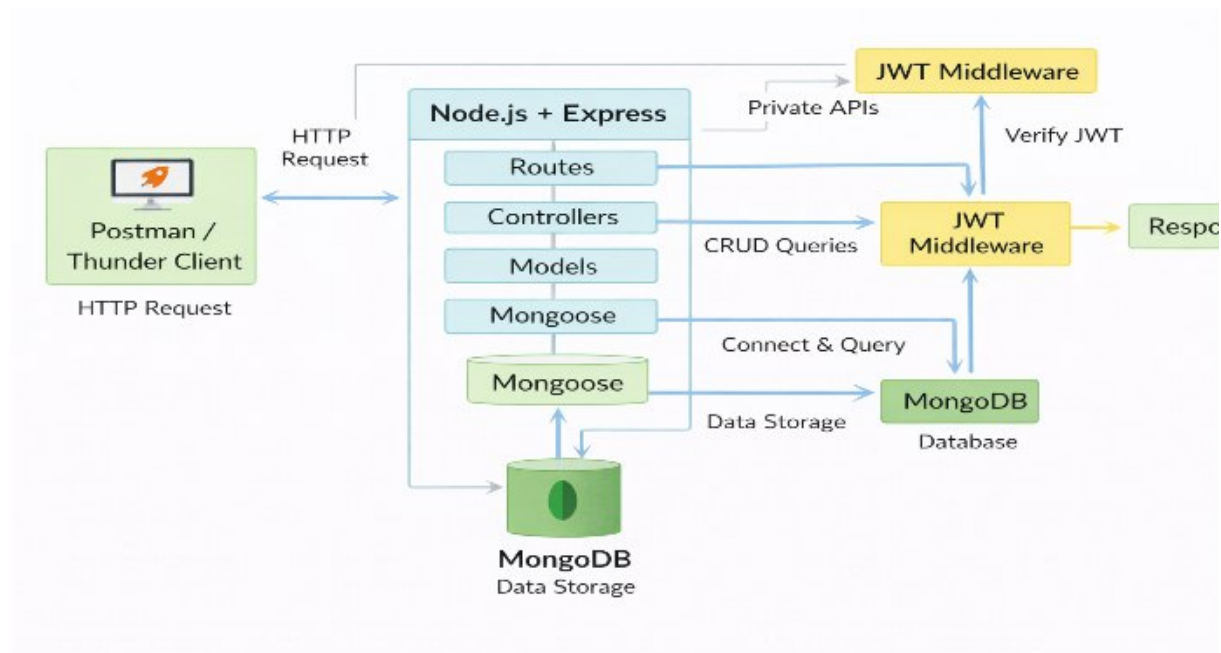
Knowledge Requirements

- JavaScript ES6+
- Node.js and Express.js
- MongoDB and Mongoose
- JWT authentication
- Basic REST API concepts
- Git basics

Thumbnail for Project

- A simple backend project thumbnail representing: Node.js + Express + MongoDB + JWT + CRUD APIs

Project Architecture Image



Technical Architecture

Backend Architecture (Node.js + Express)

- Controllers: Handle business logic for authentication and CRUD
- Models: Mongoose schemas for User and Transaction
- Routes: API endpoints for auth and CRUD operations
- Middleware: JWT verification and request protection
- Configuration: Database connection and environment variables

Database Architecture (MongoDB)

- Users Collection:

`_id, name, email, password (hashed), createdAt`

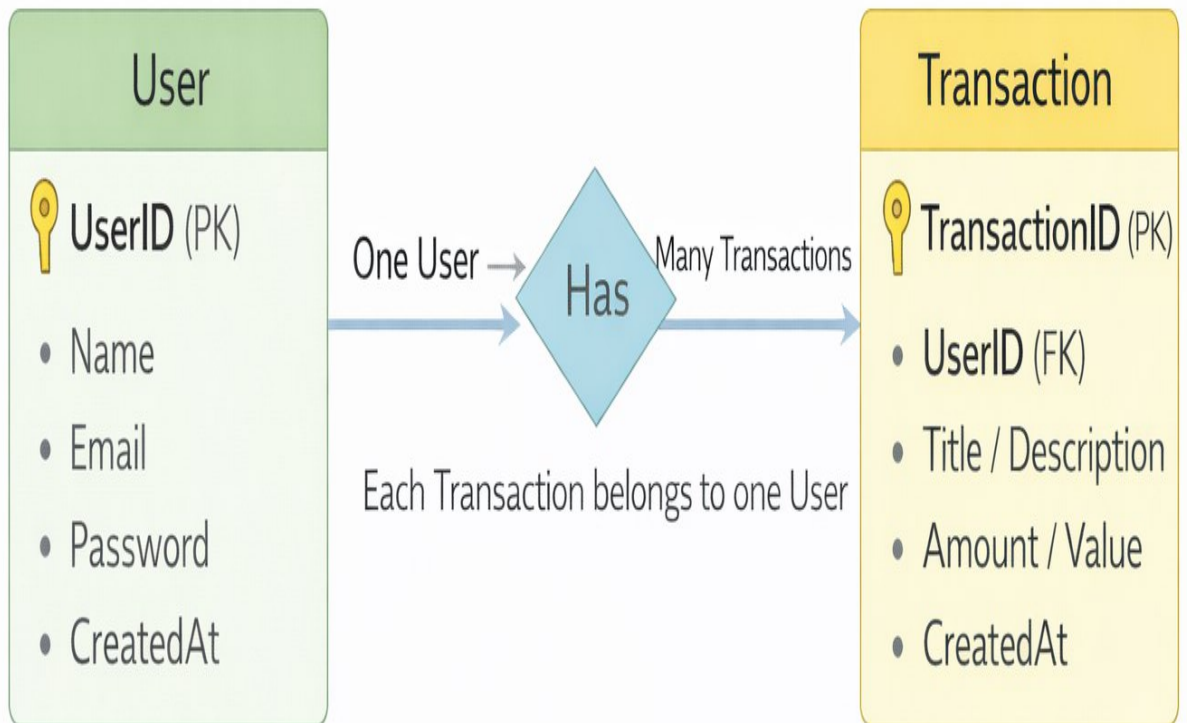
- Transactions Collection:

`_id, userId (FK), title/description, amount/value, createdAt`

- Relationship:

Many Transactions belong to One User

ER Diagram



- One User → Many Transactions
- Each Transaction belongs to one User

Features

#	Feature	Description
1	User Authentication	JWT-based registration and login
2	CRUD Operations	Create, Read, Update, Delete transactions
3	Protected Routes	Secure APIs using JWT middleware
4	MongoDB Integration	Store and fetch data using Mongoose

Roles and Responsibilities

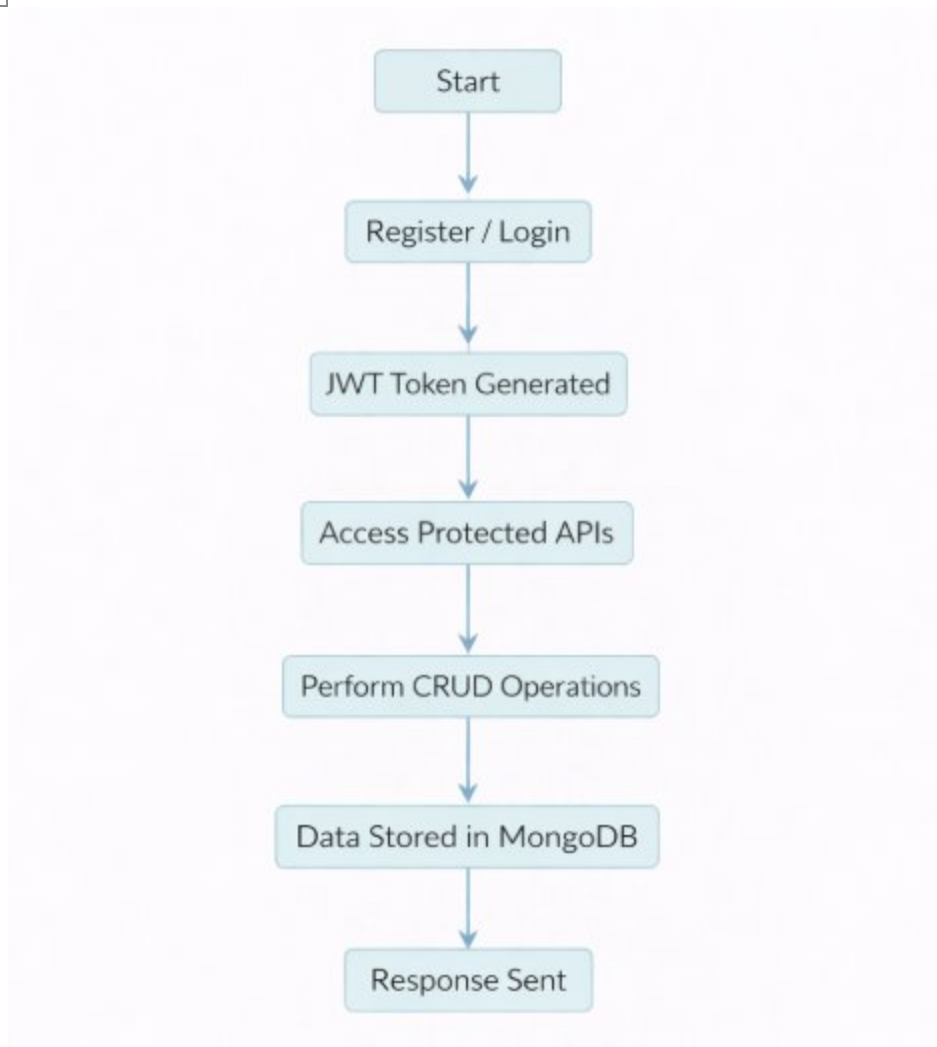
Student / Developer:

- Write backend code
- Create APIs
- Test APIs using Postman/Thunder Client
- Push code to GitHub

System (Backend):

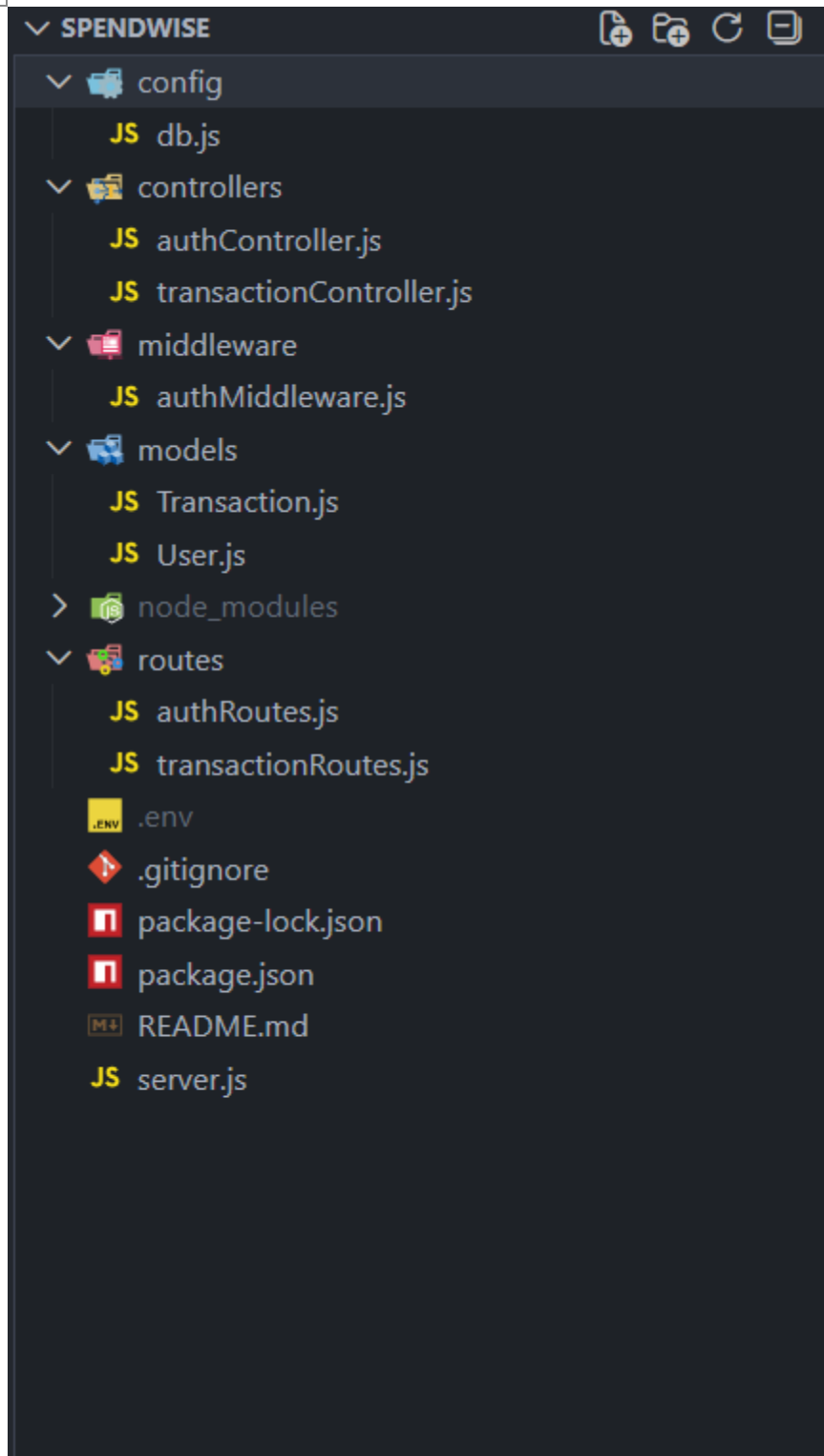
- Authenticate users using JWT
- Handle CRUD operations
- Store and fetch data from MongoDB
- Protect private routes

User's Flow:



PROJECT SETUP AND CONFIGURATION

Folder Structure



Project Set up

1. Install required tools:

- Node.js
- MongoDB

2. Create project folders:

- server folders (models, routes, controllers, middleware, config)

3. Install Backend Packages:

- Express
- Mongoose
- Cors
- Jsonwebtoken
- Bcrypt
- Dotenv

BACKEND DEVELOPMENT

- Set up Express server
- Create `server.js` file
- Create `.env` file and define PORT and DB URL

```
.env
1 PORT=5000
2 MONGO_URI=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
3 JWT_SECRET=your_secret_key
4
```

- Configure middleware (cors, express.json)
- User Authentication:
 - o Create Register and Login routes
 - o Generate JWT token
 - o Protect routes using middleware

- Define API Routes:
 - o Auth routes

```
const router = express.Router();

router.post("/register", registerUser);
router.post("/login", loginUser);

module.exports = router;
```

- Transaction CRUD routes

- Implement Data Models:

- User schema

```
const userSchema = new mongoose.Schema(  
  {  
    name: {  
      type: String,  
      required: true,  
      trim: true,  
    },  
    email: {  
      type: String,  
      required: true,  
      unique: true,  
      lowercase: true,  
      trim: true,  
    },  
    password: {  
      type: String,  
      required: true,  
      minlength: 6,  
    },  
  },  
  { timestamps: true }  
);  
  
module.exports = mongoose.model("User", userSchema);
```

- Transaction schema

```
const transactionSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    amount: {
      type: Number,
      required: true,
    },
    type: {
      type: String,
      enum: ["income", "expense"],
      required: true,
    },
    category: {
      type: String,
      default: "General",
    },
    note: {
      type: String,
    },
    date: {
      type: Date,
      default: Date.now,
    },
  },
  { timestamps: true }
);
```

- Implement CRUD operations
- Add basic error handling

DATABASE DEVELOPMENT

Configure MongoDB

- Use MongoDB Atlas or Local MongoDB
- Store connection URL in .env file
- Connect using Mongoose

Create Database Connection

- Create a config file for DB connection

```
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

config > JS db.js > ...
1  const mongoose = require("mongoose");
2
3  const connectDB = async () => {
4    try {
5      const conn = await mongoose.connect(process.env.MONGO_URI);
6      console.log(`MongoDB Connected: ${conn.connection.host}`);
7    } catch (error) {
8      console.error("MongoDB connection error:", error.message);
9      process.exit(1);
10   }
11 };
12
13 module.exports = connectDB;
```

- Use `mongoose.connect()`
- Handle success and error cases

Create Schema and Models

User Model:

- Name
- email
- password

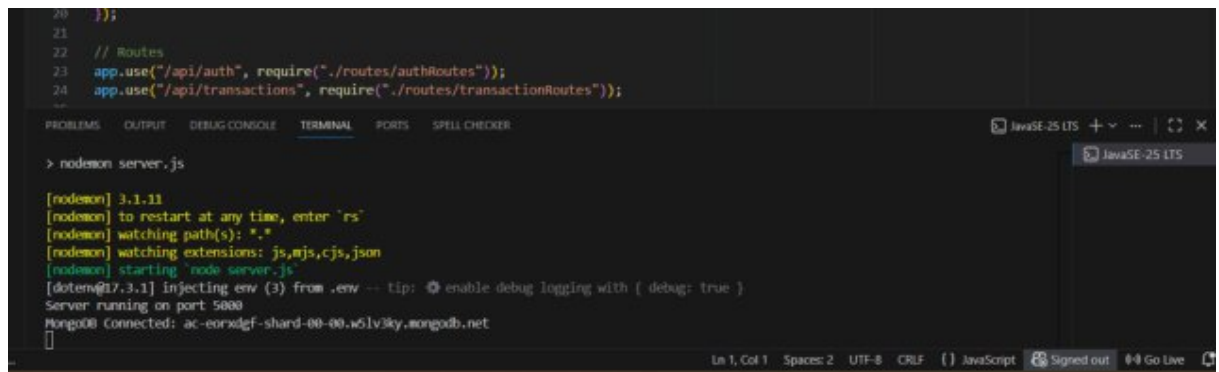
Data / Transaction Model:

- userId
- title / description
- amount / value
- timestamps

PROJECT EXECUTION

Project Execution Steps:

1. Start MongoDB
2. Run backend server



The screenshot shows a VS Code editor with a JavaScript file open. The code defines routes for authentication and transactions. Below the editor, the terminal window is active, showing the command `nodemon server.js` being executed. The terminal output indicates that nodemon is running on port 5000 and is connected to a MongoDB instance. The status bar at the bottom shows the file is named `server.js`, is 1 line long, and is using UTF-8 encoding.

```
20 });
21
22 // Routes
23 app.use("/api/auth", require("./routes/authRoutes"));
24 app.use("/api/transactions", require("./routes/transactionRoutes"));
25
> nodemon server.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node server.js'
[dotenv@17.3.1] Injecting env (3) from .env -- tip: enable debug logging with { debug: true }
Server running on port 5000
MongoDB Connected: ac-eorxldgf-shard-00-00.w6lv3ky.mongodb.net
[ ]
```

3. Test Register API

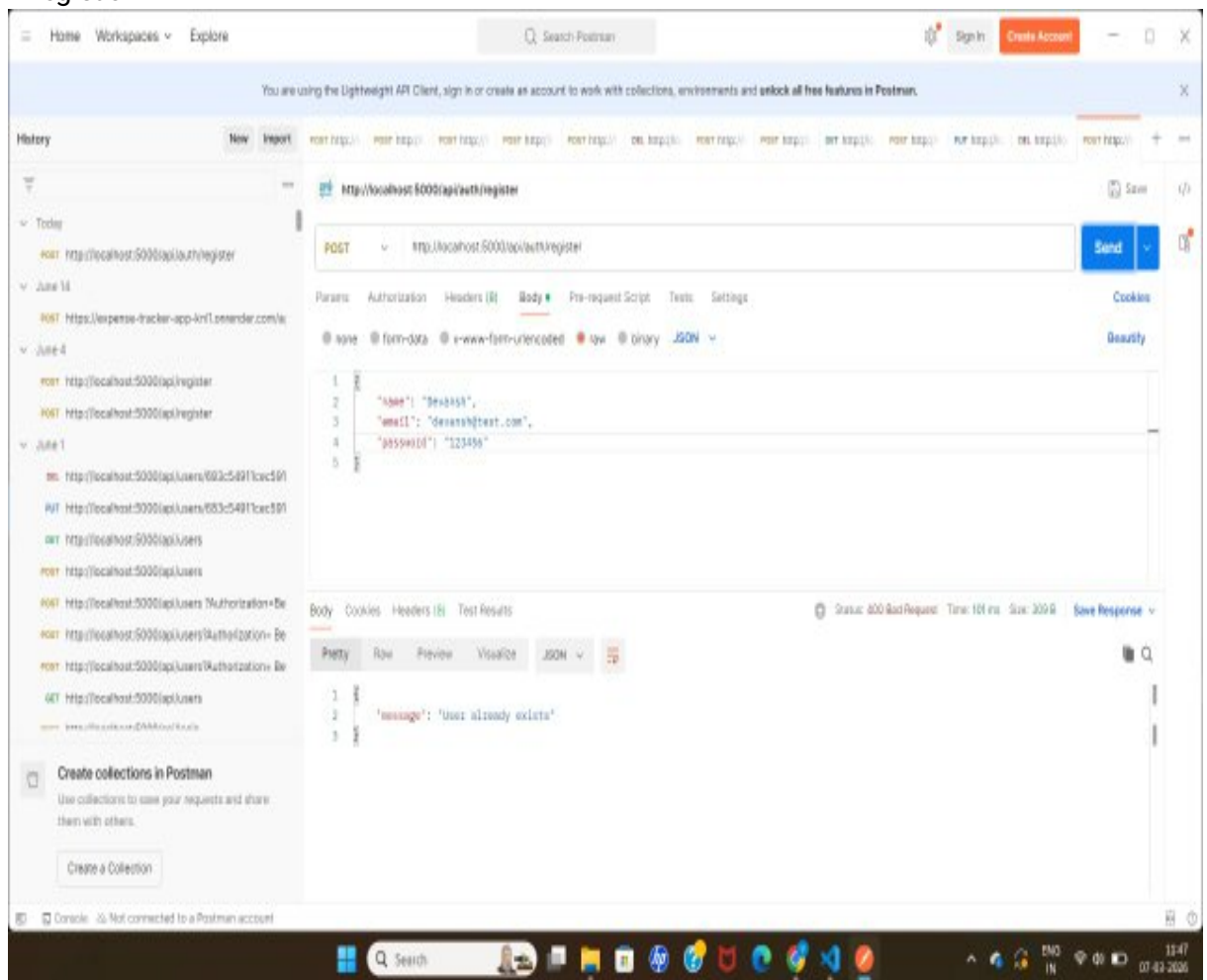


6. Use token to access protected CRUD APIs

7. Perform Create, Read, Update, Delete operations

OUTPUT SCREENSHOT

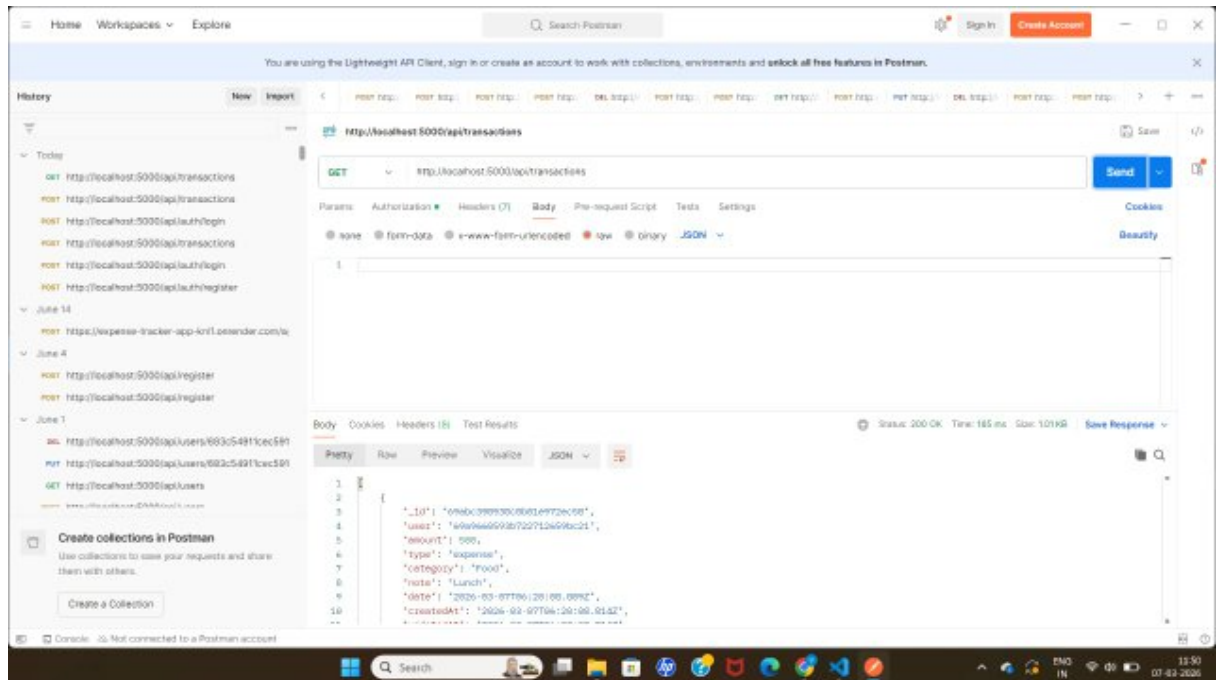
1. Register API



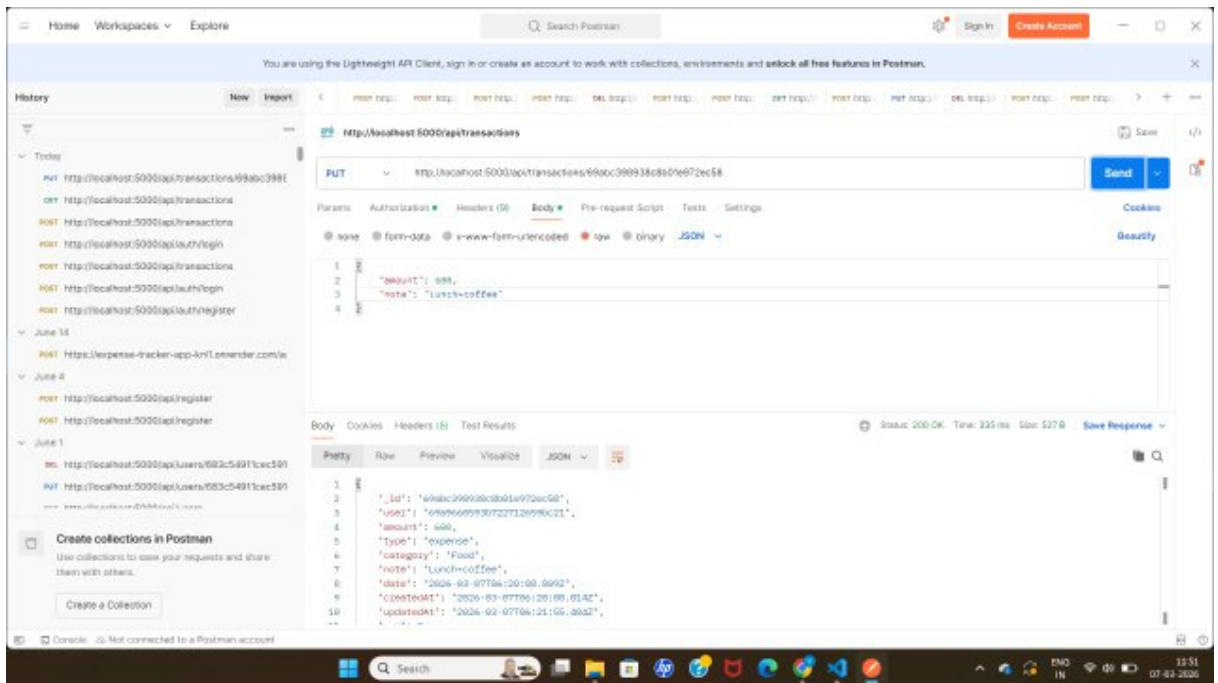
2. Login API



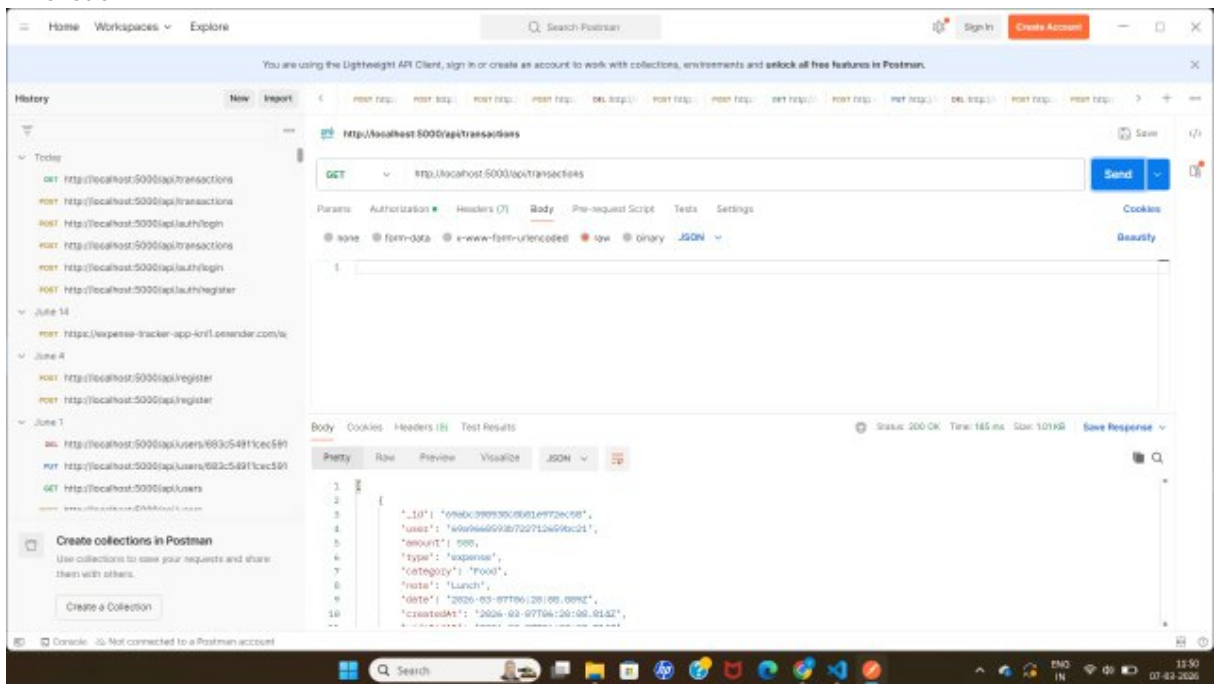
4. Get information



5. Update API



6. Delete API



Project Demo Video

<https://drive.google.com>

Drive Link : [SpendWise](https://drive.google.com)